

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

Лабораторна робота 1

з дисципліни
«Об'єктно-орієнтоване програмування»

на тему

«Знайомство із середовищем розробки програм Microsoft Visual Studio та
складання модульних проєктів програм на C++»

Виконав:

студент групи ІМ-55

Бабенко Ярослав Володимирович

Номер в списку - 3

Перевірив:

Рекечинський Д.О.

Київ 2026

Завдання:

1. Створити у середовищі MS Visual Studio C++ проєкт з ім'ям Lab1.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. Налогодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.

Завдання згідно варіанту:

B1: 1

Вікно діалогу з повзунком горизонтального скролінгу (Horizontal scroll Bar) та дві кнопки: [Так] і [Відміна]. Рухаючи повзунок скролінгу користувач вводить число у діапазоні від 1 до 100. Після натискування кнопки [Так] вибране число буде відображатися у головному вікні

B2: 0

Вікно діалогу для вводу тексту, яке має стрічку вводу (Edit Control) та дві кнопки: [Так] і [Відміна]. Якщо ввести рядок тексту і натиснути [Так], то у головному вікні повинен відображатися текст, що був введений.

Вихідний текст файлів:

Lab1.cpp

```
#include "framework.h"
#include "Lab1.h"

// Підключення інтерфейсів наших модулів
#include "module1.h"
#include "module2.h"

#define MAX_LOADSTRING 100

HINSTANCE hInst;
WCHAR szTitle[MAX_LOADSTRING];
WCHAR szWindowClass[MAX_LOADSTRING];

// Змінні для збереження результатів викликів
static int g_scrollResult = 0;
static char g_textResult[256] = "";
static int g_activeMode = 0;

ATOM      MyRegisterClass(HINSTANCE hInstance);
BOOL      InitInstance(HINSTANCE, int);
LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
INT_PTR CALLBACK About(HWND, UINT, WPARAM, LPARAM);

int APIENTRY wWinMain(_In_ HINSTANCE hInstance,
    _In_opt_ HINSTANCE hPrevInstance,
    _In_ LPWSTR lpCmdLine,
    _In_int_ nCmdShow)
{
    UNREFERENCED_PARAMETER(hPrevInstance);
    UNREFERENCED_PARAMETER(lpCmdLine);

    LoadStringW(hInstance, IDS_APP_TITLE, szTitle, MAX_LOADSTRING);
    LoadStringW(hInstance, IDC_LAB1, szWindowClass, MAX_LOADSTRING);
    MyRegisterClass(hInstance);

    if (!InitInstance(hInstance, nCmdShow))
    {
        return FALSE;
    }

    HACCEL hAccelTable = LoadAccelerators(hInstance, MAKEINTRESOURCE(IDC_LAB1));
    MSG msg;
```

```

while (GetMessage(&msg, nullptr, 0, 0))
{
    if (!TranslateAccelerator(msg.hwnd, hAccelTable, &msg))
    {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
}

return (int)msg.wParam;
}

ATOM MyRegisterClass(HINSTANCE hInstance)
{
    WNDCLASSEXW wcex;
    wcex.cbSize = sizeof(WNDCLASSEX);
    wcex.style = CS_HREDRAW | CS_VREDRAW;
    wcex.lpfnWndProc = WndProc;
    wcex.cbClsExtra = 0;
    wcex.cbWndExtra = 0;
    wcex.hInstance = hInstance;
    wcex.hIcon = LoadIcon(hInstance, MAKEINTRESOURCE(IDI_LAB1));
    wcex.hCursor = LoadCursor(nullptr, IDC_ARROW);
    wcex.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wcex.lpszMenuName = MAKEINTRESOURCEW(IDC_LAB1);
    wcex.lpszClassName = szWindowClass;
    wcex.hIconSm = LoadIcon(wcex.hInstance, MAKEINTRESOURCE(IDI_SMALL));

    return RegisterClassExW(&wcex);
}

BOOL InitInstance(HINSTANCE hInstance, int nCmdShow)
{
    hInst = hInstance;
    HWND hWnd = CreateWindowW(szWindowClass, szTitle, WS_OVERLAPPEDWINDOW,
        CW_USEDEFAULT, 0, CW_USEDEFAULT, 0, nullptr, nullptr, hInstance, nullptr);

    if (!hWnd)
    {
        return FALSE;
    }

    ShowWindow(hWnd, nCmdShow);
    UpdateWindow(hWnd);
    return TRUE;
}

LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    switch (message)
    {

```

```

case WM_COMMAND:
{
    int wmId = LOWORD(wParam);
    switch (wmId)
    {
        case ID_ACTIONS_WORK1:
            if (Func_MOD1(hWnd, g_scrollResult) == 1) {
                g_activeMode = 1;
                InvalidateRect(hWnd, NULL, TRUE);
            }
            break;

        case ID_ACTIONS_WORK2:
            if (Func_MOD2(hWnd, g_textResult, 256) == 1) {
                g_activeMode = 2;
                InvalidateRect(hWnd, NULL, TRUE);
            }
            break;

        case IDM_ABOUT:
            DialogBox(hInst, MAKEINTRESOURCE(IDD_ABOUTBOX), hWnd, About);
            break;
        case IDM_EXIT:
            DestroyWindow(hWnd);
            break;
        default:
            return DefWindowProc(hWnd, message, wParam, lParam);
    }
}
break;

case WM_PAINT:
{
    PAINTSTRUCT ps;
    HDC hdc = BeginPaint(hWnd, &ps);

    // Вивід результату першого модуля
    if (g_activeMode == 1) {
        char buf[50];
        wsprintfA(buf, "Result: %d", g_scrollResult);
        TextOutA(hdc, 50, 50, buf, lstrlenA(buf));
    }
    // Вивід результату другого модуля
    else if (g_activeMode == 2) {
        TextOutA(hdc, 50, 50, "Entered text:", 13);
        TextOutA(hdc, 150, 50, g_textResult, lstrlenA(g_textResult));
    }

    EndPaint(hWnd, &ps);
}
break;

```

```

case WM_DESTROY:
    PostQuitMessage(0);
    break;

default:
    return DefWindowProc(hWnd, message, wParam, lParam);
}
return 0;
}

INT_PTR CALLBACK About(HWND hDlg, UINT message, WPARAM wParam, LPARAM lParam)
{
    UNREFERENCED_PARAMETER(lParam);
    switch (message)
    {
    case WM_INITDIALOG:
        return (INT_PTR)TRUE;

    case WM_COMMAND:
        if (LOWORD(wParam) == IDOK || LOWORD(wParam) == IDCANCEL)
        {
            EndDialog(hDlg, LOWORD(wParam));
            return (INT_PTR)TRUE;
        }
        break;
    }
    return (INT_PTR)FALSE;
}

```

module1.cpp

```

#include "module1.h"
#include "module1.rh"
static int currentPos = 1;

static INT_PTR CALLBACK Work1(HWND hDlg, UINT iMessage, WPARAM wParam, LPARAM lParam) {
    switch (iMessage) {
    case WM_INITDIALOG:
        currentPos = 1;
        SetScrollRange(GetDlgItem(hDlg, IDC_SCROLLBAR1), SB_CTL, 1, 100, TRUE);
        SetScrollPos(GetDlgItem(hDlg, IDC_SCROLLBAR1), SB_CTL, currentPos, TRUE);
        SetDlgItemInt(hDlg, IDC_STATIC_VAL, currentPos, FALSE);
        return (INT_PTR)TRUE;

    case WM_HSCROLL: {
        int pos = GetScrollPos(GetDlgItem(hDlg, IDC_SCROLLBAR1), SB_CTL);
        switch (LOWORD(wParam)) {
            case SB_LINELEFT: pos--; break;
            case SB_LINERIGHT: pos++; break;
            case SB_THUMBTRACK: pos = HIWORD(wParam); break;
        }
    }
    }
}

```

```

    }
    if (pos < 1) pos = 1;
    if (pos > 100) pos = 100;
    SetScrollPos(GetDlgItem(hDlg, IDC_SCROLLBAR1), SB_CTL, pos, TRUE);
    SetDlgItemInt(hDlg, IDC_STATIC_VAL, pos, FALSE);
    currentPos = pos;
    return (INT_PTR)TRUE;
}
case WM_COMMAND:
    if (LOWORD(wParam) == IDOK) {
        EndDialog(hDlg, 1);
        return (INT_PTR)TRUE;
    }
    if (LOWORD(wParam) == IDCANCEL) {
        EndDialog(hDlg, 0);
        return (INT_PTR)TRUE;
    }
    break;
}
return (INT_PTR)FALSE;
}

int Func_MOD1(HWND hWnd, int& outValue) {
    HINSTANCE hInst = (HINSTANCE)GetWindowLongPtr(hWnd, GWLP_HINSTANCE);
    int result = DialogBox(hInst, MAKEINTRESOURCE(IDD_DIALOG1), hWnd, Work1);
    if (result == 1) {
        outValue = currentPos;
    }
    return result;
}

```

module2.cpp

```

#include "module2.h"
#include "module2.rh"

static char* g_outText = nullptr;
static int g_maxLen = 0;

static INT_PTR CALLBACK Work2(HWND hDlg, UINT iMessage, WPARAM wParam, LPARAM lParam) {
    switch (iMessage) {
        case WM_INITDIALOG:
            SetDlgItemTextA(hDlg, IDC_EDIT1, "");
            return (INT_PTR)TRUE;

        case WM_COMMAND:
            if (LOWORD(wParam) == IDOK) {
                GetDlgItemTextA(hDlg, IDC_EDIT1, g_outText, g_maxLen);
            }
    }
}

```

```

        EndDialog(hDlg, 1);
        return (INT_PTR)TRUE;
    }
    if (LOWORD(wParam) == IDCANCEL) {
        EndDialog(hDlg, 0);
        return (INT_PTR)TRUE;
    }
    break;
}
return (INT_PTR)FALSE;
}

int Func_MOD2(HWND hWnd, char* outText, int maxlen) {
    g_outText = outText;
    g_maxLen = maxlen;
    HINSTANCE hInst = (HINSTANCE)GetWindowLongPtr(hWnd, GWLP_HINSTANCE);
    return DialogBox(hInst, MAKEINTRESOURCE(IDD_DIALOG2), hWnd, Work2);
}

```

module1.h

```

#pragma once
#include <windows.h>

```

```

extern int Func_MOD1(HWND hWnd, int& outValue);

```

module1.rh

```

#pragma once
#define IDD_DIALOG1 1301
#define IDC_SCROLLBAR1 1302
#define IDC_STATIC_VAL 1303

```

module2.h

```

#pragma once
#include <windows.h>

```

```

extern int Func_MOD2(HWND hWnd, char* outText, int maxlen);

```

module2.rh

```

#pragma once
#define IDD_DIALOG2 1401
#define IDC_EDIT1 1402

```

module1.rc

```
#include "windows.h"
#include "module1.rh"

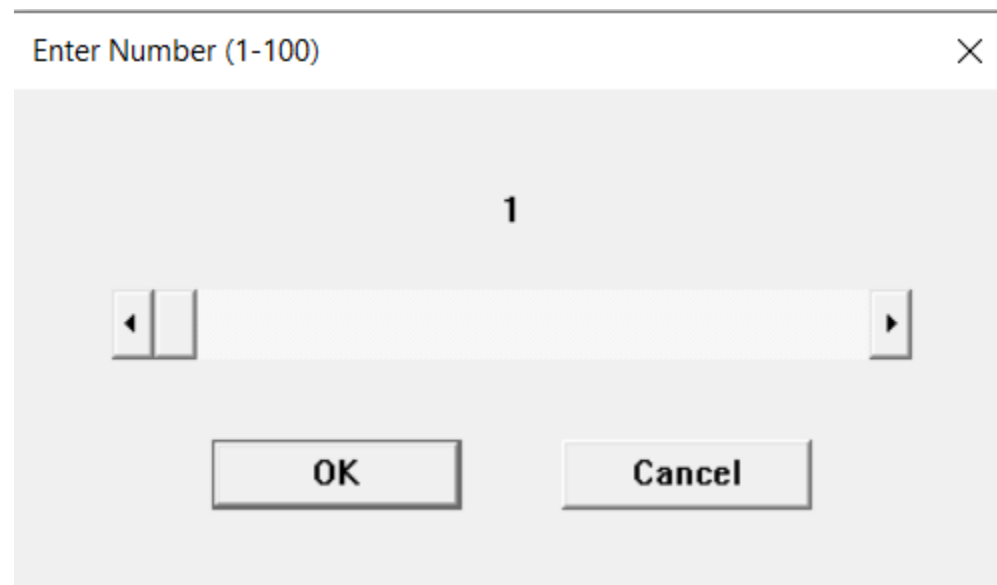
IDD_DIALOG1 DIALOGEX 0, 0, 200, 100
STYLE DS_MODALFRAME | WS_POPUP | WS_CAPTION | WS_SYSMENU
CAPTION "Enter Number (1-100)"
BEGIN
    DEFPUSHBUTTON  "OK", IDOK, 40, 70, 50, 14
    PUSHBUTTON     "Cancel", IDCANCEL, 110, 70, 50, 14
    SCROLLBAR      IDC_SCROLLBAR1, 20, 40, 160, 14, SBS_HORZ
    CTEXT          "1", IDC_STATIC_VAL, 80, 20, 40, 12, SS_CENTER
END
```

module2.rc

```
#include "windows.h"
#include "module2.rh"

IDD_DIALOG2 DIALOGEX 0, 0, 200, 80
STYLE DS_MODALFRAME | WS_POPUP | WS_CAPTION | WS_SYSMENU
CAPTION "Enter Text"
BEGIN
    EDITTEXT       IDC_EDIT1, 20, 20, 160, 14, ES_AUTOHSCROLL
    DEFPUSHBUTTON  "OK", IDOK, 40, 50, 50, 14
    PUSHBUTTON     "Cancel", IDCANCEL, 110, 50, 50, 14
END
```

Скріншоти



Enter Number (1-100) ×

40

◀ ▶

OK Cancel

📁 Lab1

Файл Справка Завдання

Result: 40

Enter Text ×

привіт

OK Cancel

 Lab1

Файл Справка Завдання

Entered text: привіт

Висновки:

У лабораторній роботі я виконав завдання згідно свого варіанту. Під час виконання лабораторної роботи було успішно розроблено модульний проєкт, який реалізує принципи об'єктно-орієнтованого програмування.

Виконуючи це завдання, я здобув важливі практичні навички та поглибив розуміння сучасної розробки. Отримані навички є міцним фундаментом для подальшого проєктування складних застосунків та глибокого засвоєння об'єктно-орієнтованого програмування.